

Intelligent Plant Disease Detection System

Project Overview Report

98.85%

Test Accuracy

42

Disease
Classes

50,400

Balanced
Images

5-
Fold

Cross
Validation

University	Misr University for Science and Technology
Supervisor	Dr. Heba ELnemr
Date	April 2026
Model	ResNet50 — PyTorch — Google Colab (NVIDIA L4)

PROJECT TEAM

Yousef Ellawah

Omar Walid

Mohamed Emad

Nour Mohamed

Menna Mohamed

Ahmed Abdul-Wahab

For educational and research purposes only

1. Project Description

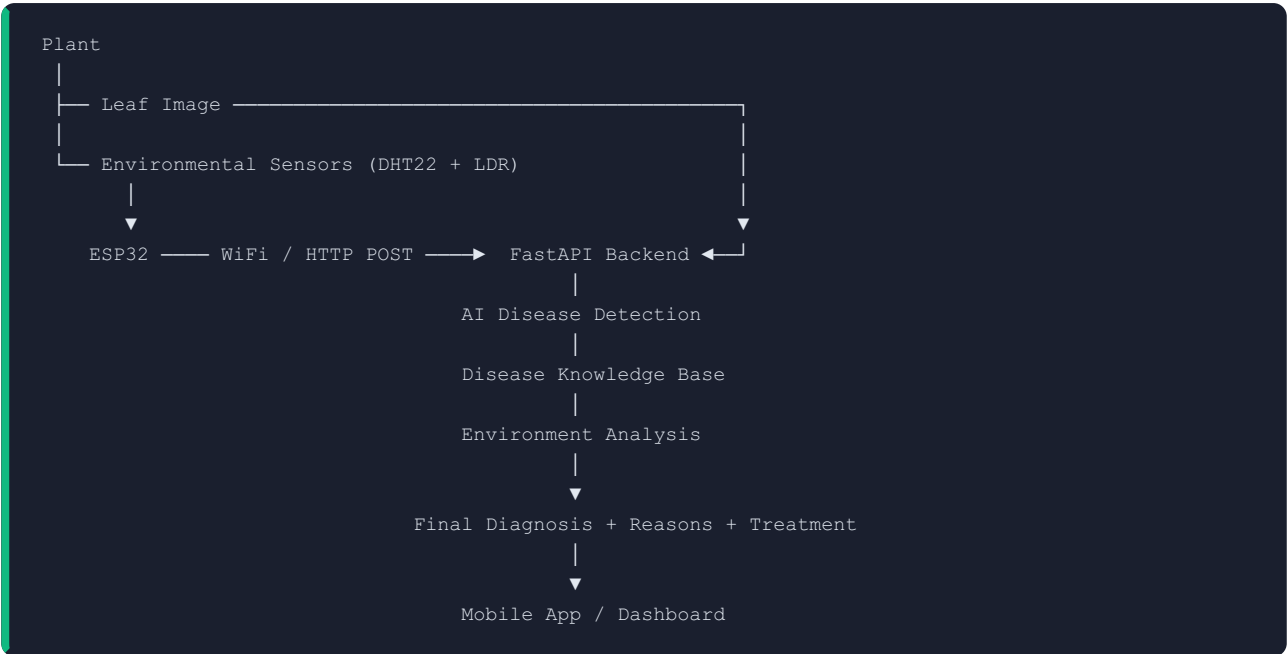
This project is an AI-powered smart agriculture system for early plant disease detection. It goes beyond simple image classification by combining deep learning with real-time IoT environmental monitoring to provide a complete diagnostic experience.

The system takes a leaf image captured by the user, analyzes it using a trained ResNet50 model, and simultaneously reads environmental data (temperature, humidity, light) from sensors placed near the plant. It then combines both results to explain possible causes and provide targeted treatment recommendations — all displayed in Arabic and English through a mobile application.

Core Components

Component	Role
AI Model (ResNet50)	Analyzes leaf image → identifies disease and confidence
ESP32 + Sensors	Reads temperature, humidity, and light intensity in real time
FastAPI Backend	Integrates AI prediction with sensor data and disease knowledge base
Disease Knowledge Base	42-class bilingual database with symptoms, treatments, and prevention
Flutter Mobile App	Displays full diagnosis to the user (Android · iOS · Web · Desktop)

System Flow



2. Datasets

2.1 Sources

Dataset	Description	Classes	Images
PlantVillage	Widely used benchmark dataset for plant disease classification	38	~71,307
RiceLeafs_merged_224	Additional rice leaf disease images (224×224)	4	~6,000
Total (balanced)	Both datasets merged and balanced to 1,200 images per class	42	50,400

2.2 Supported Crops & Diseases (42 Classes)

class_id	Class Name	Crop	Status
0	Apple___Apple_scab	Apple	Diseased
1	Apple___Black_rot	Apple	Diseased
2	Apple___Cedar_apple_rust	Apple	Diseased
3	Apple___healthy	Apple	Healthy
4	Blueberry___healthy	Blueberry	Healthy
5	Cherry_(including_sour)___Powdery_mildew	Cherry	Diseased
6	Cherry_(including_sour)___healthy	Cherry	Healthy
7	Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot	Corn	Diseased
8	Corn_(maize)___Common_rust_	Corn	Diseased
9	Corn_(maize)___Northern_Leaf_Blight	Corn	Diseased
10	Corn_(maize)___healthy	Corn	Healthy
11	Grape___Black_rot	Grape	Diseased
12	Grape___Esca_(Black_Measles)	Grape	Diseased
13	Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	Grape	Diseased
14	Grape___healthy	Grape	Healthy
15	Orange___Haunglongbing_(Citrus_greening)	Orange	Diseased
16	Peach___Bacterial_spot	Peach	Diseased
17	Peach___healthy	Peach	Healthy
18	Pepper,_bell___Bacterial_spot	Bell Pepper	Diseased
19	Pepper,_bell___healthy	Bell Pepper	Healthy
20	Potato___Early_blight	Potato	Diseased
21	Potato___Late_blight	Potato	Diseased
22	Potato___healthy	Potato	Healthy
23	Raspberry___healthy	Raspberry	Healthy
24	Rice___Brown_spot	Rice	Diseased
25	Rice___Healthy	Rice	Healthy
26	Rice___Hispa	Rice	Diseased
27	Rice___Leaf_blast	Rice	Diseased

class_id	Class Name	Crop	Status
28	Soybean___healthy	Soybean	Healthy
29	Squash___Powdery_mildew	Squash	Diseased
30	Strawberry___Leaf_scorch	Strawberry	Diseased
31	Strawberry___healthy	Strawberry	Healthy
32	Tomato___Bacterial_spot	Tomato	Diseased
33	Tomato___Early_blight	Tomato	Diseased
34	Tomato___Late_blight	Tomato	Diseased
35	Tomato___Leaf_Mold	Tomato	Diseased
36	Tomato___Septoria_leaf_spot	Tomato	Diseased
37	Tomato___Spider_mites Two-spotted_spider_mite	Tomato	Diseased
38	Tomato___Target_Spot	Tomato	Diseased
39	Tomato___Tomato_Yellow_Leaf_Curl_Virus	Tomato	Diseased
40	Tomato___Tomato_mosaic_virus	Tomato	Diseased
41	Tomato___healthy	Tomato	Healthy

29 diseased classes · 13 healthy classes · 15 crop types

2.3 Class Imbalance Handling

The original datasets have significant class imbalance (e.g., Tomato has ~5,000 images while Raspberry has ~370). To address this, each class was balanced to **1,200 images per fold**:

- Classes with **fewer than 1,200** images → Oversampled with replacement
- Classes with **more than 1,200** images → Randomly undersampled without replacement

3. AI Model — Training

3.1 Architecture

Property	Value
Architecture	ResNet50
Pretrained Weights	IMAGENET1K_V2
Fine-tuning	All layers (full fine-tuning)
Output Layer	nn.Linear(2048 → 42)
Input Size	224 × 224 px
Framework	PyTorch + Torchvision
Platform	Google Colab · NVIDIA L4

3.2 Data Split

```
Full Dataset (50,400 images — balanced)
├──
├── Test Set — 20% (10,080 images) — locked, never seen during training
├──
└── Train + Validation — 80% (40,320 images)
    ├──
    └── StratifiedKfold (5 Folds)
        ├── Fold 1: ~32,256 train / ~8,064 val
        ├── Fold 2: ~32,256 train / ~8,064 val
        ├── Fold 3: ~32,256 train / ~8,064 val
        ├── Fold 4: ~32,256 train / ~8,064 val ★ Best
        └── Fold 5: ~32,256 train / ~8,064 val
```

3.3 Training Configuration

Parameter	Value
Optimizer	AdamW
Learning Rate	1e-4
Weight Decay	1e-4
Scheduler	CosineAnnealingLR
Loss Function	CrossEntropyLoss (label_smoothing=0.05)
Batch Size	32
Max Epochs	30
Early Stopping	patience=5 · min_delta=0.001
TTA Steps	5
Seed	42

3.4 Augmentation

Training: RandomResizedCrop · RandomHorizontalFlip · RandomVerticalFlip · RandomRotation(30°) · ColorJitter · GaussianBlur · RandomPerspective · RandomGrayscale · RandomErasing · ImageNet Normalize

Validation / Test: Resize(224×224) · ImageNet Normalize

Test-Time Augmentation (TTA): Applied 5 times per image on final evaluation; predictions averaged to improve accuracy.

3.5 Cross-Validation Results

Fold	Val Accuracy	Val F1	Stop Epoch	Note
Fold 1	98.34%	98.34%	19	Very strong
Fold 2	98.83%	98.84%	22	Excellent
Fold 3	97.46%	97.47%	11	Lowest fold
Fold 4	98.87%	98.88%	29	★ Best — selected as final model
Fold 5	98.77%	98.77%	29	Excellent
Mean	98.45% ± 0.60%	98.46%	—	

3.6 Final Test Results

Evaluated on the locked test set of **10,080 samples** using TTA:

Metric	Value
Test Accuracy	98.85%
Macro Precision	98.86%
Macro Recall	98.85%
Macro F1-Score	98.85%

28 out of 42 classes achieved perfect Precision = Recall = F1 = 1.00.

3.7 Hardest Classes

The rice classes were the most challenging due to visually similar leaf patterns:

Class	Precision	Recall	F1-Score
Rice___Healthy	0.82	0.87	0.85
Rice___Hispa	0.89	0.84	0.86
Rice___Leaf_blast	0.92	0.92	0.92

4. Hardware

4.1 Components

Component	Model	Purpose
Microcontroller	ESP32-D0WD-V3 (ESP32 Dev Module)	Reads sensors and sends data via WiFi
Temperature & Humidity	DHT22 Module	Measures temperature (°C) and humidity (%)
Light Sensor	LDR Module (4-pin)	Measures ambient light intensity (analog)
Wiring	Breadboard + Jumper Wires	Prototyping and connections
Power	USB Cable from Laptop / Adapter	Powers and programs the ESP32

Note: The board appears in Arduino IDE as **ESP32-D0WD-V3**. Select **ESP32 Dev Module** (not ESP32S3) in the Board Manager.

4.2 Wiring

Power Distribution:

ESP32 Pin	Breadboard
3V3	+ rail
GND	– rail

DHT22 (3-pin module):

DHT22 Pin	Connection
VCC	3.3V
GND	GND
DAT	GPIO 4

LDR Module (4-pin) — use AO only:

LDR Pin	Connection
VCC	3.3V
GND	GND
AO	GPIO 34
DO	Not connected

4.3 Sensor Data

The ESP32 reads sensors every 2 seconds and sends data to the backend:

```
{
  "temperature": 26.0,
  "humidity": 56.9,
  "light": 294
}
```

Light reading correction — the raw LDR value is inverted (dark = high, bright = low), so it is corrected in firmware:

```
int fixedLight = 4095 - rawLight;
int lightPercent = map(rawLight, 4095, 0, 0, 100);
```

4.4 Arduino IDE Setup

1. Install Arduino IDE
2. Add ESP32 board URL in Preferences: `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package\_esp32\_index.json`
3. Install **esp32 by Espressif Systems** from Boards Manager
4. Select **ESP32 Dev Module** from Tools → Board
5. Install **DHT sensor library by Adafruit + Adafruit Unified Sensor** from Library Manager
6. Upload `hardware/esp32_sensor_sender.ino`

Full wiring guide → `plant hardware setup.md`

5. Backend API

Built with **FastAPI** · runs locally or on any server · model loaded at startup.

Endpoints

Method	Endpoint	Description
GET	/	Health check
POST	/predict	Send leaf image → returns disease, confidence, top-3, env analysis
POST	/sensor-data	ESP32 pushes sensor reading
GET	/sensor-data	Get latest sensor reading

Predict Response

```
{
  "class_id": 33,
  "disease": "Tomato__Early_blight",
  "confidence": 0.9741,
  "top3": [
    { "disease": "Tomato__Early_blight", "confidence": 0.9741 },
    { "disease": "Tomato__Late_blight", "confidence": 0.0142 },
    { "disease": "Tomato__Target_Spot", "confidence": 0.0081 }
  ],
  "sensor_data": { "temperature": 28.5, "humidity": 65, "light": 1850 },
  "env_analysis": {
    "environmental_risk": "high",
    "temperature_status": "favorable",
    "humidity_status": "favorable",
    "light_status": "normal",
    "can_improve_by_env": true,
    "summary_en": "Current environment is contributing to disease spread.",
    "improvement_tips_en": [
      "Ensure adequate sunlight – remove lower leaves for better airflow.",
      "Use mulch to prevent soil splash onto leaves."
    ]
  }
}
```

Running the Backend

```
cd plant_mobile_app
python3 -m venv venv
source venv/bin/activate
pip install -r backend/requirements.txt
cd backend
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

6. Disease Knowledge Base

Overview

The file `data/diseases_database.csv` (and `.json`) is a bilingual (Arabic + English) knowledge base containing all 42 classes used by the model.

Property	Value
Rows	42 (one per class)
Columns	25
Languages	Arabic + English
Diseased classes	29
Healthy classes	13
Crop types	15

Key Columns

Column	Description
<code>class_id</code> / <code>class_name</code>	Match model output exactly
<code>crop_en</code> / <code>crop_ar</code>	Crop name (bilingual)
<code>disease_en</code> / <code>disease_ar</code>	Disease name (bilingual)
<code>status</code>	<code>healthy</code> or <code>diseased</code>
<code>pathogen_en</code> / <code>pathogen_ar</code>	Causal organism
<code>symptoms_en</code> / <code>symptoms_ar</code>	Visual symptoms
<code>environmental_factors_en/ar</code>	Climate/environment triggers
<code>predisposing_stress_en/ar</code>	Stress factors that increase susceptibility
<code>chemical_treatment_en/ar</code>	Chemical control options
<code>organic_treatment_en/ar</code>	Organic/biological control options
<code>prevention_en/ar</code>	Prevention guidance
<code>severity</code>	<code>Low</code> / <code>Medium</code> / <code>High</code>
<code>season_en</code> / <code>season_ar</code>	Typical season of occurrence
<code>source_url</code>	Trusted reference (Cornell, Penn State, IRRI, UC IPM, etc.)

How It Works

After the model returns a `class_name` (e.g., `Apple__Black_rot`), the backend looks up the matching row in the database and retrieves symptoms, treatments, and prevention steps. It then compares the current sensor readings against the disease's known environmental triggers to generate a contextual explanation and actionable recommendations.

7. Mobile Application

Built with **Flutter** — supports Android, iOS, Web, and Desktop from a single codebase.

Features

- Capture or upload a leaf image (camera or gallery)
- Sends image to backend for AI prediction
- Displays disease name, confidence bar, and top-3 predictions
- Shows full diagnostic profile: description, symptoms, chemical & organic treatment, prevention
- Displays real-time environmental data from ESP32 sensors
- Shows environmental risk analysis and improvement tips
- Scan history
- Full Arabic / English localization with RTL support
- Light / Dark theme
- Firebase Authentication (email + password + guest mode)

Key Files

File	Description
<code>lib/main.dart</code>	App entry point, MaterialApp setup
<code>lib/ui/app_shell.dart</code>	Bottom navigation (Scan · History · Settings)
<code>lib/ui/screens/home_screen.dart</code>	Image capture / gallery picker
<code>lib/ui/screens/preview_screen.dart</code>	Image preview + trigger prediction
<code>lib/ui/screens/result_screen.dart</code>	Full diagnosis result display
<code>lib/ui/screens/history_screen.dart</code>	Past scan history
<code>lib/data/prediction_api.dart</code>	HTTP client for <code>/predict</code>
<code>lib/domain/disease_info.dart</code>	Local disease knowledge base
<code>lib/core/settings_controller.dart</code>	Theme and language settings

Running the App

```
cd mobile_app
flutter pub get
flutter run
```

For a physical device, update `baseUrl` in `lib/ui/screens/preview_screen.dart` :

```
final _api = PredictionApi(baseUrl: Uri.parse('http://192.168.1.XXX:8000'));
```

8. Technology Stack

Layer	Technologies
AI Model	Python · PyTorch · Torchvision · ResNet50
Backend	FastAPI · Uvicorn · Pillow · Pydantic
Mobile	Flutter · Dart · Firebase Auth · image_picker · http
Hardware	ESP32 · DHT22 · LDR · Arduino IDE · C++
Database	CSV / JSON disease knowledge base
Training	Google Colab · NVIDIA L4 · 5-Fold Cross Validation

9. Project Phases

Phase	Description	Status
1 — AI Model	Dataset preparation, training, evaluation	✔ Complete
2 — Application	Mobile app UI/UX, backend API, web dashboard	✔ Complete
3 — Hardware	ESP32 sensors, environmental monitoring, IoT integration	✔ Complete

10. Academic Context

Field	Details
University	Misr University for Science and Technology (MUST)
Supervisor	Dr. Heba ELnemr
Project Title	Intelligent Plant Disease Detection Using Deep Learning
Year	2026

References

- Schmidt et al. (2022). *An Analysis of the Accuracy of Photo-Based Plant Identification Applications*. *Arboriculture & Urban Forestry*, 48(1). [doi:10.48044/jauf.2022.003](https://doi.org/10.48044/jauf.2022.003)
- Siddiqua et al. (2022). *Evaluating Plant Disease Detection Mobile Applications*. *Agronomy*, 12(8). [doi:10.3390/agronomy12081869](https://doi.org/10.3390/agronomy12081869)
- Ibrahim et al. (2025). *AI-IoT based smart agriculture pivot for plant diseases detection*. *Scientific Reports*, 15(1). [doi:10.1038/s41598-025-98454-6](https://doi.org/10.1038/s41598-025-98454-6)
- Nyakuri et al. (2025). *AI and IoT-powered edge device for crop pest and disease detection*. *Scientific Reports*, 15(1). [doi:10.1038/s41598-025-06452-5](https://doi.org/10.1038/s41598-025-06452-5)
- Khanal et al. (2024). *Paddy Disease detection and classification using Computer Vision*. *arXiv*. [arxiv:2412.05996](https://arxiv.org/abs/2412.05996)
- Dhaka et al. (2023). *Role of IoT and Deep Learning in Plant Disease Detection*. *Sensors*, 23(18). [doi:10.3390/s23187877](https://doi.org/10.3390/s23187877)
- Ahmed & Reddy (2021). *A Mobile-Based system for detecting plant leaf diseases*. *AgriEngineering*, 3(3). [doi:10.3390/agriengineering3030032](https://doi.org/10.3390/agriengineering3030032)
- Zhao et al. (2024). *APEIOU Integration for Enhanced YOLOv7: Efficient Plant Disease Detection*. *Agriculture*, 14(6). [doi:10.3390/agriculture14060820](https://doi.org/10.3390/agriculture14060820)